

Système de Gestion de Stock

Application Web Django

Réalisé par : AMLLAL Amine
Encadré par : M. Brahim Bakkas
Date : 20 avril 2026

Objectif du projet : Concevoir et implémenter une application web professionnelle de gestion de stock, avec gestion des produits et des catégories, contrôle d'accès par rôles, intégration d'images, et tableaux de bord visuels orientés exploitation.

Table des matières

1	Introduction	3
2	Analyse des besoins	3
2.1	Besoins fonctionnels	3
2.2	Besoins non fonctionnels	3
3	Description fonctionnelle	3
3.1	Profils utilisateurs	3
3.2	Fonctionnalités principales	4
4	Architecture technique	4
4.1	Vue d'ensemble	4
4.2	Sécurité et contrôle d'accès	4
5	Technologies utilisées	4
6	Structure du code	4
6.1	Organisation du dépôt	4
6.2	Fichiers principaux et rôles	5
7	Logique de fonctionnement et workflow	5
7.1	Chaîne fonctionnelle principale	5
7.2	Règles de stock	5
8	Implémentation	5
8.1	Couche modèle	6
8.2	Couche vue/contrôleur	6
8.3	Couche présentation	6
8.4	Données de démonstration	6
9	Tests et validation	6
9.1	Validation technique	6
9.2	Limites actuelles	6
10	Exploitation du dossier docs	6
10.1	Captures d'écran de l'application	7

10.1.1	Vue administrateur	7
10.1.2	Vue utilisateur standard	8
10.2	Diagrammes UML	8
10.2.1	Diagramme de cas d'utilisation	9
10.2.2	Diagramme de classes	10
10.2.3	Diagramme de composants	11
10.2.4	Diagramme de séquence : consultation de la liste produits	12
10.2.5	Diagramme de séquence : déconnexion	13
10.2.6	Diagramme d'activité : ajout produit	14
10.2.7	Diagramme d'états : statut de stock	15
11	Conclusion et perspectives	15

1 Introduction

La gestion des stocks constitue un besoin central pour toute structure manipulant des produits physiques. Le projet réalisé met en place une application web Django permettant de centraliser la consultation, la création, la modification et la suppression d'articles, tout en intégrant une gouvernance claire des droits utilisateur.

L'application vise un compromis entre simplicité d'usage, lisibilité de l'information et robustesse technique. Elle s'appuie sur un socle Django 5.x, une base SQLite pour le développement, une interface Bootstrap 5.3 et une logique de permissions fondée sur les groupes `superadmin`, `admin` et `viewer`.

2 Analyse des besoins

2.1 Besoins fonctionnels

Le système doit couvrir les besoins suivants :

- gestion des produits (CRUD) avec photo, prix, quantité et catégorie ;
- gestion des catégories (CRUD) ;
- consultation filtrée et triée du catalogue ;
- pagination des résultats ;
- visualisation de l'état de stock (disponible, faible, rupture) ;
- authentification et contrôle d'accès par profils ;
- messages de retour utilisateur pour chaque action métier à impact base de données.

2.2 Besoins non fonctionnels

Les exigences non fonctionnelles principales sont :

- interface responsive et claire ;
- configuration locale Windows simple ;
- code maintenable et organisé par responsabilités ;
- compatibilité multilingue française (locale `fr-fr`) ;
- traçabilité des opérations via messages utilisateurs.

3 Description fonctionnelle

3.1 Profils utilisateurs

Le projet implémente quatre niveaux de droits :

- **Superuser Django** : accès complet à l'administration et à l'application ;
- **Superadmin** : gestion produits + catégories ;
- **Admin** : gestion produits uniquement ;
- **Viewer** : consultation uniquement.

3.2 Fonctionnalités principales

- liste produits avec recherche textuelle, filtre par catégorie, tri multi-critères et pagination ;
- fiche détaillée d'un produit ;
- formulaires de création et modification produits/catégories ;
- confirmation explicite avant suppression ;
- déconnexion sécurisée par requête POST avec jeton CSRF.

4 Architecture technique

4.1 Vue d'ensemble

L'architecture suit le patron Django MVT (Model-View-Template) :

- **Models** : représentation des entités `Category` et `Product` ;
- **Views** : logique applicative, contrôles d'accès, requêtes et orchestration ;
- **Templates** : rendu HTML Bootstrap côté client.

Le routage global est géré par `gestion_stock/urls.py`, qui inclut les routes métier de `inventory/urls.py` et la livraison des médias en mode DEBUG.

4.2 Sécurité et contrôle d'accès

Les fonctions `is_superuser` et `can_manage_products` centralisent les règles d'autorisation. Les vues critiques utilisent `@login_required` et `@user_passes_test`. La déconnexion est implémentée via POST pour respecter les bonnes pratiques HTTP/CSRF.

5 Technologies utilisées

Composant	Choix technique
Langage	Python 3.11+
Framework web	Django 5.2.13
Base de données	SQLite (<code>db.sqlite3</code>)
Gestion images	Pillow 12.2.0 + stockage local <code>media/products</code>
UI	Bootstrap 5.3 via CDN
Templates	Django Template Engine
OS cible développement	Windows (PowerShell)

6 Structure du code

6.1 Organisation du dépôt

- `gestion_stock/` : configuration Django (settings, urls, wsgi/asgi) ;

- `inventory/` : logique métier (models, views, forms, admin, urls, seed command) ;
- `templates/inventory/` : templates HTML ;
- `media/` : ressources uploadées (photos produits) ;
- `docs/` : captures d'écran et diagrammes UML.

6.2 Fichiers principaux et rôles

Fichier	Rôle
<code>inventory/models.py</code>	Définition des entités et logique <code>stock_status</code> .
<code>inventory/views.py</code>	Contrôle des flux métier, permissions, CRUD, recherche/tri/pagination.
<code>inventory/forms.py</code>	Formulaires ModelForm et widgets Bootstrap.
<code>inventory/urls.py</code>	Routage des cas d'usage produits/catégories/logout.
<code>templates/inventory/base.html</code>	Layout global, navigation, messages, actions par rôle.
<code>inventory/management/commands/seed.py</code>	Génération de données de démonstration et comptes.
<code>gestion_stock/settings.py</code>	Paramètres globaux (locale FR, timezone, media, auth).

7 Logique de fonctionnement et workflow

7.1 Chaîne fonctionnelle principale

1. L'utilisateur s'authentifie via `/admin/login/`.
2. La page d'accueil affiche la liste paginée des produits.
3. Les filtres GET `q`, `category`, `sort` affinent le résultat.
4. Les rôles `admin/superadmin` accèdent aux actions CRUD produit.
5. Le rôle `superadmin` accède en plus au CRUD catégorie.
6. Les opérations sensibles affichent un message de succès/alerte.
7. La déconnexion ferme la session et redirige vers la page de connexion.

7.2 Règles de stock

Le statut de stock est calculé dynamiquement :

- `out` si `stock == 0` ;
- `low` si `stock <= 5` ;
- `available` sinon.

Ce statut pilote les badges de couleur dans l'interface (rouge, jaune, vert).

8 Implémentation

8.1 Couche modèle

La classe `Category` structure le référentiel catégories (nom unique, description, date de création). La classe `Product` relie chaque produit à une catégorie avec une contrainte de suppression en cascade, ce qui maintient la cohérence du catalogue.

8.2 Couche vue/contrôleur

La vue `product_list` utilise `select_related('category')` pour optimiser les accès relationnels, puis combine recherche, filtre, tri et pagination. Les vues d'édition/suppression sont protégées par des décorateurs d'autorisation.

8.3 Couche présentation

Les templates reposent sur un layout unique, avec une barre de navigation conditionnelle selon les droits. Les formulaires exploitent les composants Bootstrap et les pages de confirmation réduisent les suppressions accidentelles.

8.4 Données de démonstration

La commande `seed_mock_data` initialise 5 catégories, 12 produits et les groupes utilisateurs. Elle facilite les démonstrations fonctionnelles et les tests visuels.

9 Tests et validation ---

9.1 Validation technique

Les validations exécutées incluent :

- `python manage.py check` : cohérence de configuration ;
- `python manage.py makemigrations` et `migrate` : cohérence schéma ;
- vérification des workflows CRUD et des permissions par profil.

9.2 Limites actuelles

Le fichier `inventory/tests.py` ne contient pas encore de tests unitaires automatisés. La validation actuelle est principalement manuelle (tests fonctionnels par rôle et par parcours).

10 Exploitation du dossier docs ---

Cette section intègre exhaustivement l'ensemble des ressources du dossier `docs` : 2 captures d'écran et 7 diagrammes UML.

10.1 Captures d’écran de l’application

10.1.1 Vue administrateur

La Figure 1 montre la vue catalogue avec droits admin/superadmin : actions « Détail », « Modifier », « Supprimer », bouton « Ajouter produit »et filtres complets.

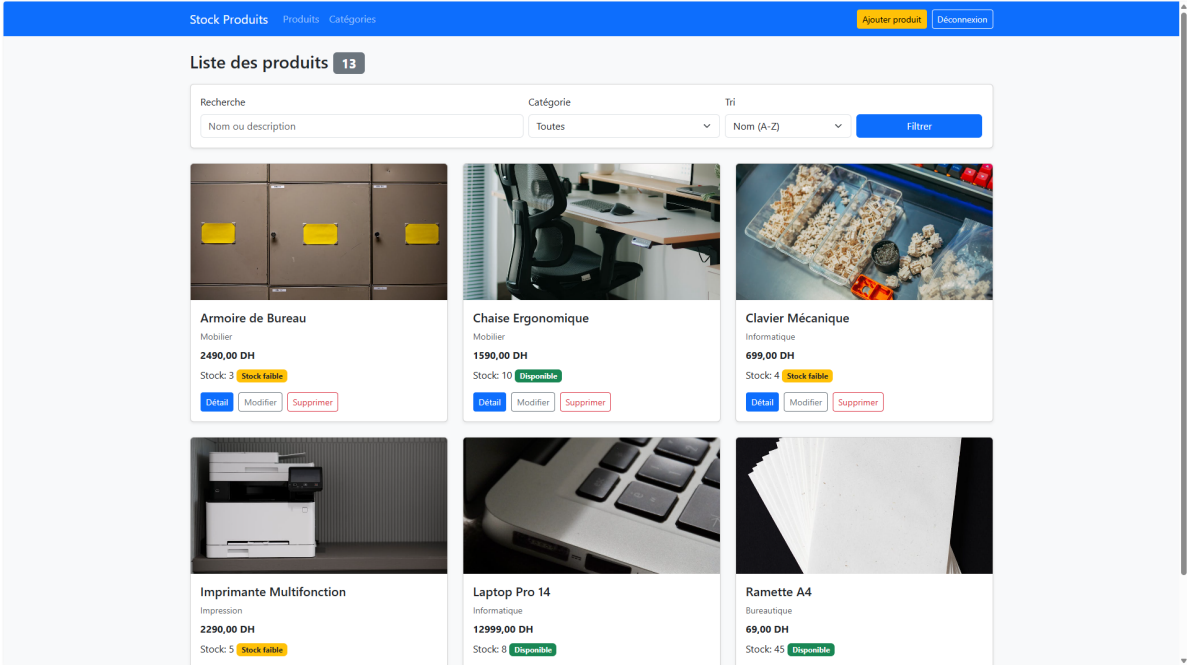


FIGURE 1 – Interface administrateur : gestion complète du catalogue produits

10.1.2 Vue utilisateur standard

La Figure 2 met en évidence la restriction des droits pour le profil **viewer** : consultation seule, sans opérations d'édition/suppression.

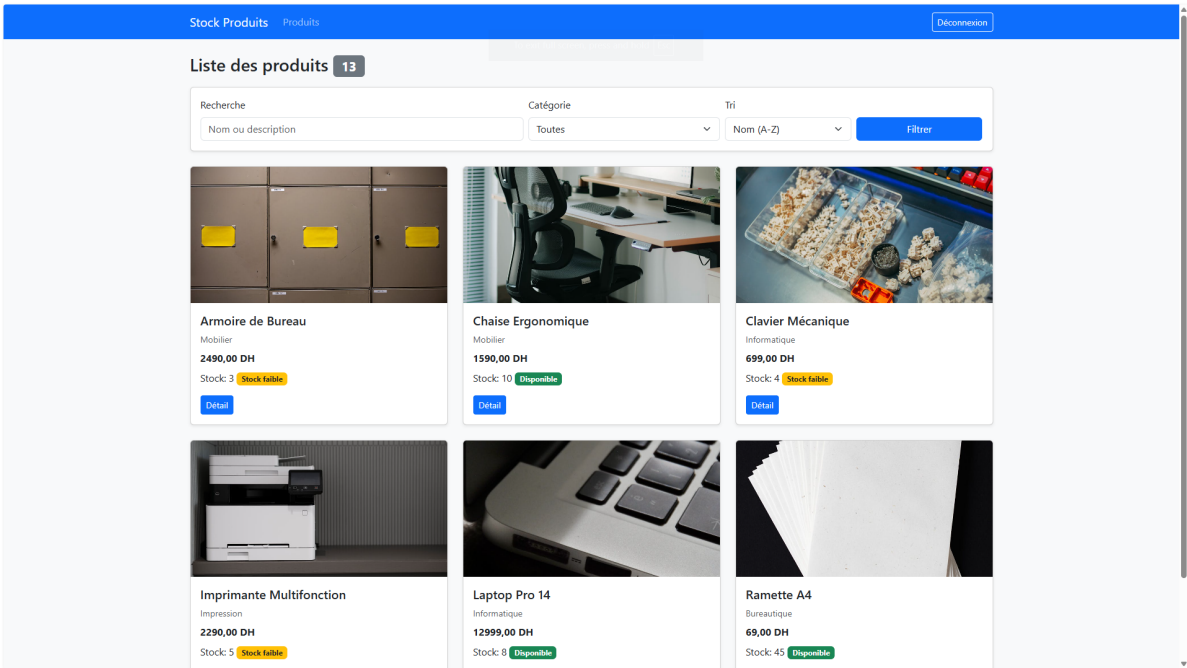


FIGURE 2 – Interface utilisateur standard : consultation sans droits de gestion

10.2 Diagrammes UML

10.2.1 Diagramme de cas d'utilisation

Fichier source docs : docs/diagrammes UML/Diagramme de cas d'utilisation.svg

Ce diagramme (Figure 3) formalise les interactions entre acteurs et fonctionnalités du système. Il joue un rôle de cadrage fonctionnel en visualisant la couverture des besoins par profil. Son interprétation confirme que la plateforme distingue clairement les capacités de consultation et de gestion.

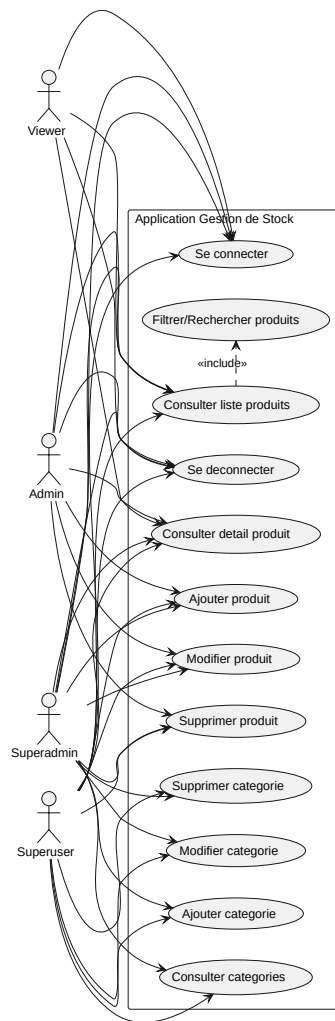


FIGURE 3 – Diagramme UML de cas d'utilisation

10.2.2 Diagramme de classes

Fichier source docs : docs/diagrammes UML/Diagramme de classes.svg

Le diagramme de classes (Figure 4) représente la structure statique : entités **Category** et **Product**, relation à cardinalité 1–N, et éléments de couche formulaire/contrôle. Il sert à vérifier la cohérence de modélisation et l’alignement avec l’implémentation Django.

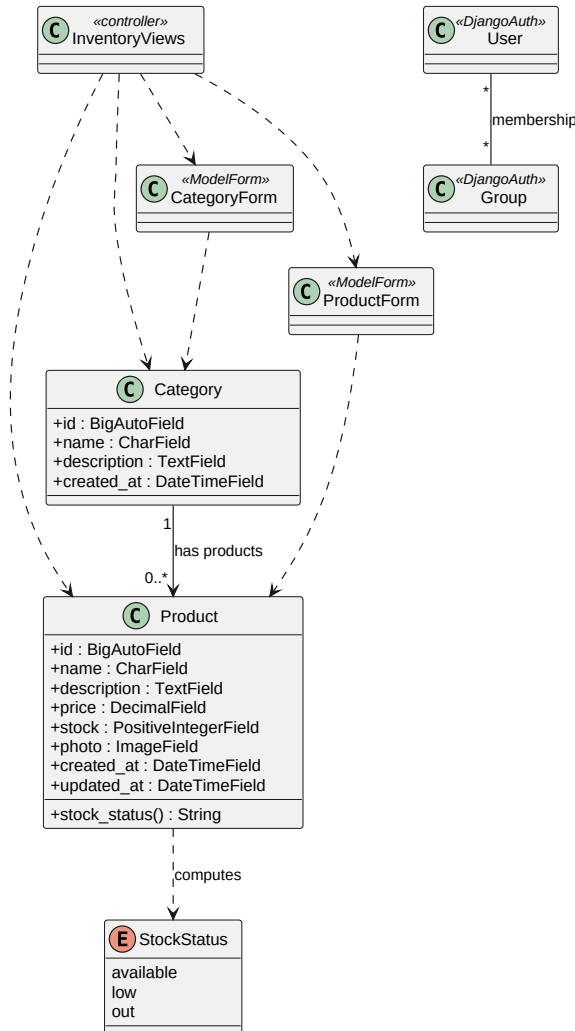


FIGURE 4 – Diagramme UML de classes

10.2.3 Diagramme de composants

Fichier source docs : docs/diagrammes UML/Diagramme de composants (07_component_diagram.puml).sv

Le diagramme de composants (Figure 5) explicite l'architecture en couches : navigateur, routage, vues, formulaires, modèles, templates, base de données et répertoire média. Il relie directement la conception globale au découpage réel du dépôt.

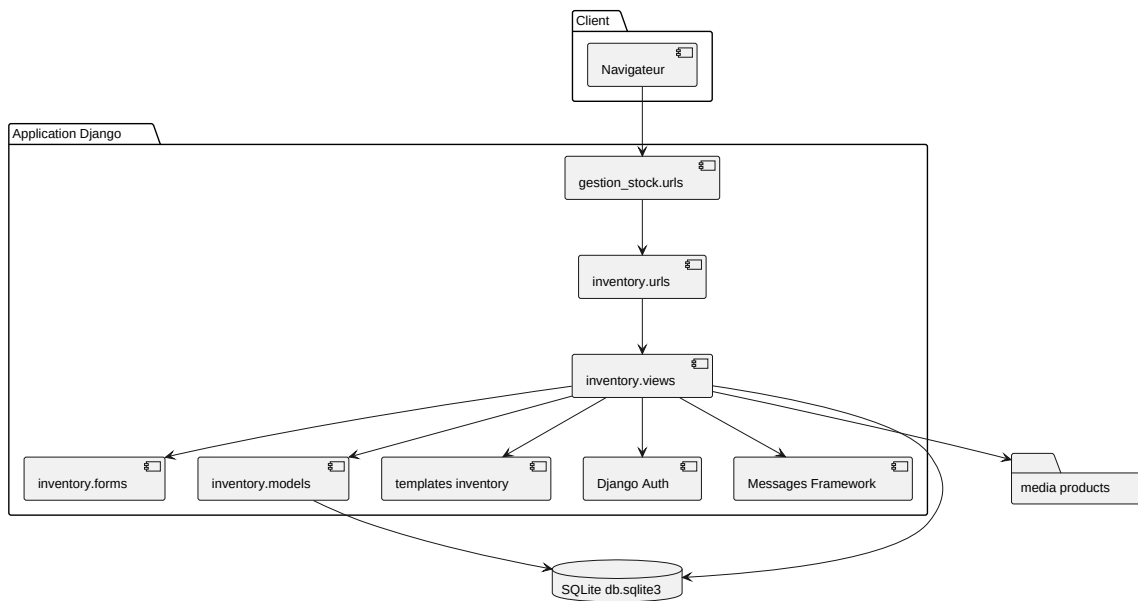


FIGURE 5 – Diagramme UML de composants

10.2.4 Diagramme de séquence : consultation de la liste produits

Fichier source docs : docs/diagrammes UML/Diagramme de sequence consultation liste produits.svg

La Figure 6 détaille la dynamique d’une requête de consultation : entrée HTTP, traitement des filtres, interrogation SQL et rendu template. Ce diagramme est essentiel pour comprendre la chaîne de performance (notamment `select_related` et pagination).

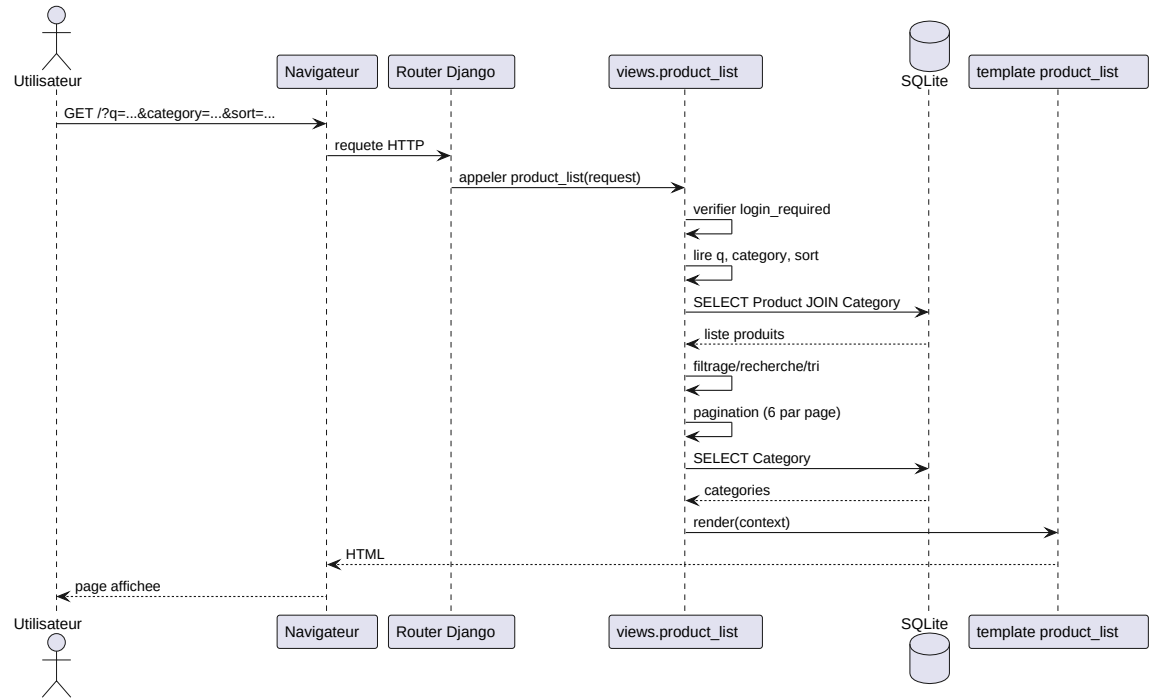


FIGURE 6 – Diagramme UML de séquence - consultation liste produits

10.2.5 Diagramme de séquence : déconnexion

Fichier source docs : docs/diagrammes UML/Diagramme de sequence deconnexion (04_sequence_logout.

Le diagramme de séquence de déconnexion (Figure 7) décompose la fermeture de session via POST, l’invalidation de session Django et la redirection vers la page de connexion. Il formalise la correction appliquée pour éliminer les erreurs HTTP 405.

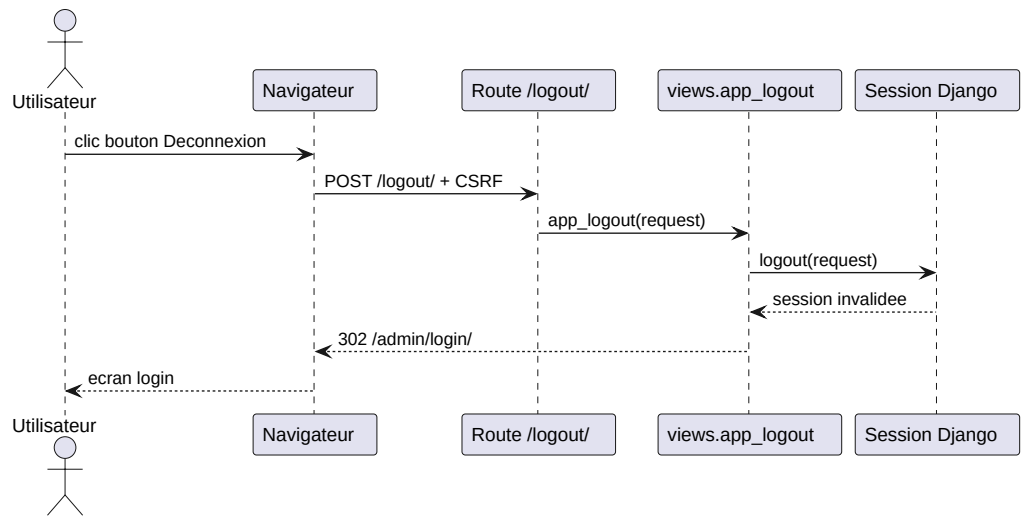


FIGURE 7 – Diagramme UML de séquence - déconnexion

10.2.6 Diagramme d'activité : ajout produit

Fichier source docs : docs/diagrammes UML/Diagramme d'activite ajout produit (05_activity_add_pr

La Figure 8 modélise le flux de création d'un produit avec points de décision : authentification, autorisation, validation formulaire, persistance et retour utilisateur. Ce diagramme relie la logique métier aux contraintes de sécurité.

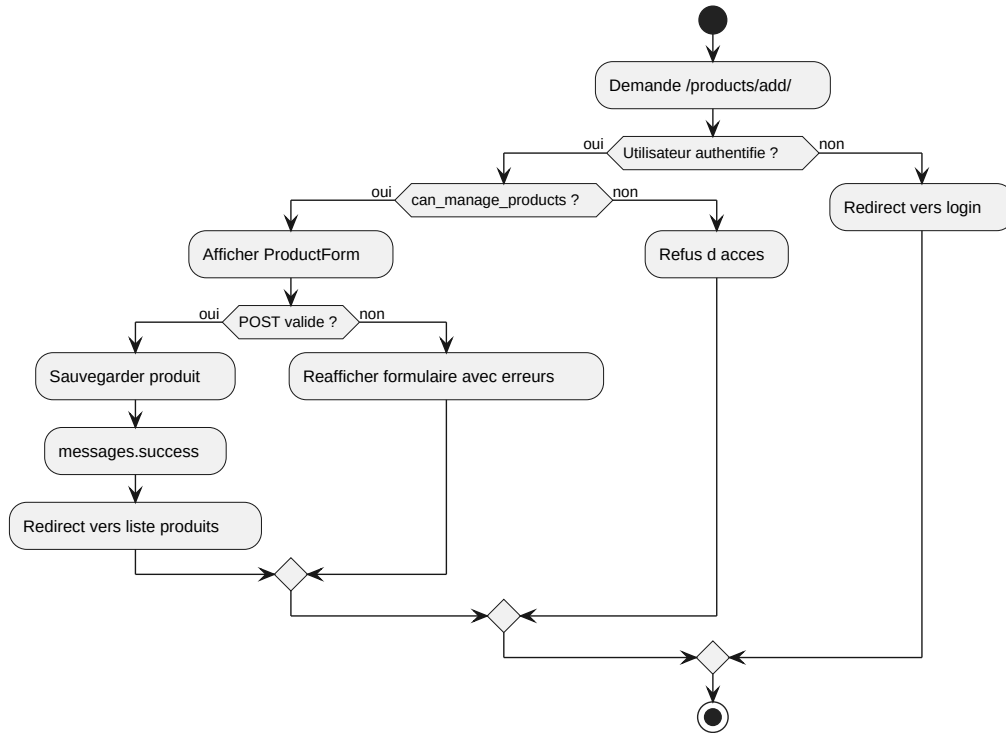


FIGURE 8 – Diagramme UML d'activité - ajout produit

10.2.7 Diagramme d'états : statut de stock

Fichier source docs : docs/diagrammes UML/Diagramme d'états corrigé (stock_status).svg

Le diagramme d'états (Figure 9) représente les transitions `available`, `low`, `out` selon la quantité. Il ancre visuellement la règle de gestion implémentée dans la propriété `stock_status` du modèle `Product`.

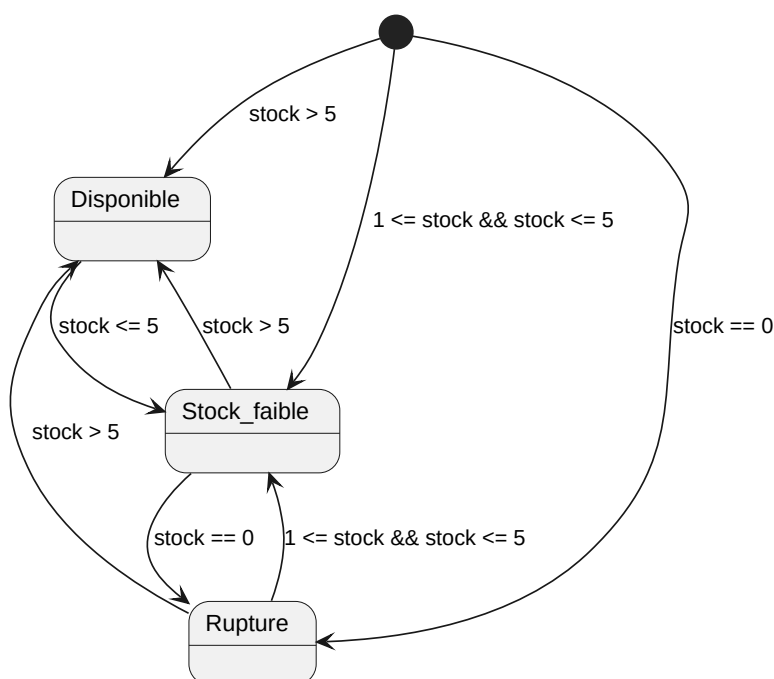


FIGURE 9 – Diagramme UML d'états - statut de stock

11 Conclusion et perspectives

Le projet aboutit à une application de gestion de stock opérationnelle, structurée et adaptée à un usage de démonstration ou de prototypage avancé. Les objectifs fonctionnels sont couverts : gestion du catalogue, filtrage, tri, pagination, rôles et visualisation de l'état de stock.

Sur le plan technique, le code est lisible, segmenté et extensible. L'intégration de données de démonstration, de ressources visuelles et de diagrammes UML renforce la qualité documentaire du projet.

Perspectives d'évolution :

- ajout de tests unitaires et d'intégration automatisés ;
- migration vers PostgreSQL pour la production ;
- historisation des mouvements de stock (entrées/sorties) ;
- ajout d'un tableau de bord analytique (statistiques, alertes) ;
- mise en place d'une API REST (Django REST Framework) pour intégration externe.